**TO-DO LIST APPLICATION USING PYTHON**

**AIM**

To design and develop an interactive To-Do List application using Python Tkinter that allows users to type tasks, add them to a visual list, delete completed tasks, and clear all tasks. The program includes a graphical user interface (GUI) with text entry fields, listboxes, buttons, and error-handling message boxes.

**PROCEDURE**

1. Start the program and import the required Python libraries: tkinter and messagebox.
2. Create the main application window using Tkinter, set the title to "To-Do List", and define window dimensions and background colors.
3. Design the graphical interface by adding a Title Label at the top of the window.
4. Create an Entry widget (text box) for the user to type their new tasks into.
5. Create a Listbox widget to visually display the added tasks.
6. Create three interactive buttons: "Add Task", "Delete Task", and "Clear All".
7. Define the add_task() function to get text from the entry box and insert it into the Listbox. If the box is empty, show a warning message.
8. Define the delete_task() function to identify the selected item in the Listbox and remove it.
9. Define the clear_all() function to delete every entry currently in the Listbox.
10. Use the .pack() method to organize and align all widgets on the screen with proper spacing.
11. Execute the root.mainloop() method to run the application and keep the window open.

**PROGRAM CODE**

```
import tkinter as tk
from tkinter import messagebox


# --- Functions for the buttons ---


def add_task():
    task = task_entry.get()
    if task != "":
        task_listbox.insert(tk.END, task)
        task_entry.delete(0, tk.END) # Clear the text box after adding
    else:
        messagebox.showwarning("Warning", "You must enter a task!")


def delete_task():
    try:
        selected_task_index = task_listbox.curselection()[0]
        task_listbox.delete(selected_task_index)
    except IndexError:
        messagebox.showwarning("Warning", "Please select a task to delete!")


def clear_all():
    task_listbox.delete(0, tk.END)
```

```
# --- Main Window Setup ---

root = tk.Tk()
root.title("TO-DO LIST APP")
root.geometry("350x450")
root.resizable(False, False)
root.configure(bg="#0F172A") # Dark blue background

# --- GUI Elements ---

# 1. Title Label
title_label = tk.Label(
    root,
    text="MY TO-DO LIST",
    font=("Arial Black", 18),
    bg="#0F172A",
    fg="#38BDF8"
)
title_label.pack(pady=15)

# 2. Text Entry Box
task_entry = tk.Entry(
    root,
    font=("Arial", 14),
    width=24,
    bg="#1E293B",
    fg="white",
    insertbackground="white" # Cursor color)
```

```
task_entry.pack(pady=10)

# 3. Add Task Button
add_button = tk.Button(
    root,
    text="ADD TASK",
    font=("Arial", 12, "bold"),
    bg="#22C55E",
    fg="black",
    command=add_task
)
add_button.pack(pady=5)

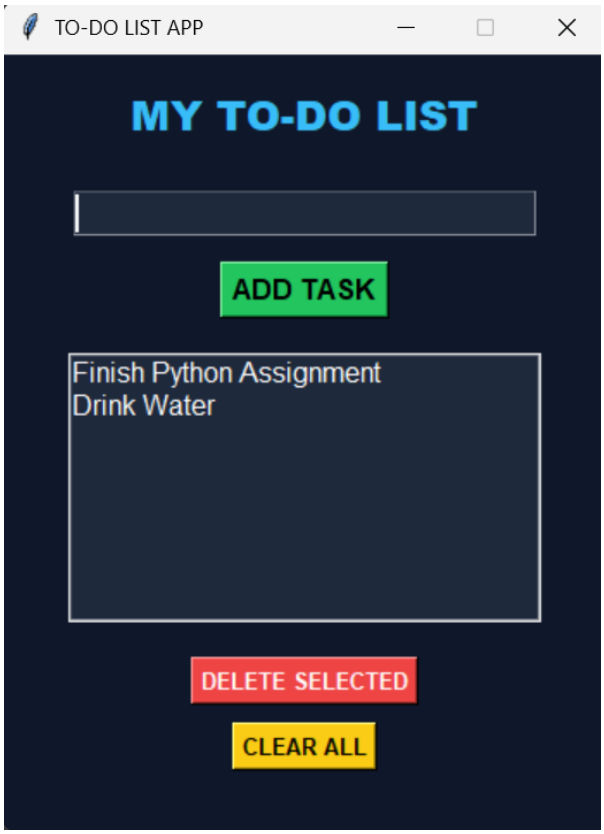
# 4. The Listbox (Shows the tasks)
task_listbox = tk.Listbox(
    root,
    font=("Arial", 12),
    width=30,
    height=8,
    bg="#1E293B",
    fg="white",
    selectbackground="#38BDF8"
)
task_listbox.pack(pady=15)
```

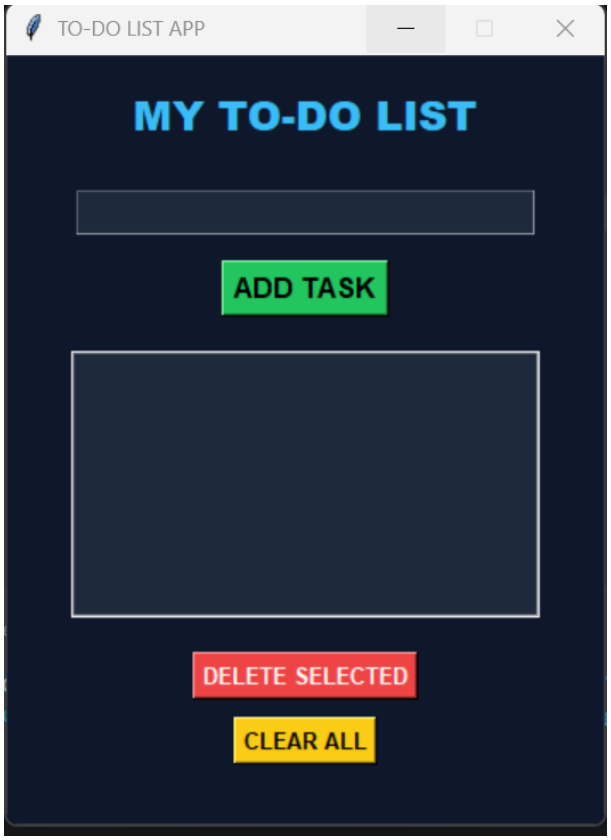
```
# 5. Delete and Clear Buttons
delete_button = tk.Button(
    root,
    text="DELETE SELECTED",
    font=("Arial", 10, "bold"),
    bg="#EF4444",
    fg="white",
    command=delete_task
)
delete_button.pack(pady=5)

clear_button = tk.Button(
    root,
    text="CLEAR ALL",
    font=("Arial", 10, "bold"),
    bg="#FACC15",
    fg="black",
    command=clear_all
)
clear_button.pack(pady=5)

# --- Run the App ---
root.mainloop()
```

**OUTPUT:**




**RESULT:**

Thus, an interactive To-Do List application was successfully developed using Python Tkinter. The program demonstrates the use of graphical user interface (GUI) design, user input handling, listbox manipulation, function definition, and event-driven programming. The application allows users to seamlessly add, view, and delete items with error handling integrated via message boxes.